

**[Sistema de gestión integral]
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	
Proyecto	Sistema de gestión integral
Versión	1.2

Fecha de última revisión	16/05/2026
Fecha de próxima revisión	18/05/2026

Documento aprobado por		
Fecha de última aprobación		

Historia de Revisiones

Fecha	Versión	Descripción	Autor
28/04/2026	1.0	prototipo inicial	
04/05/2026	1.0.1	mejoras añadidas	
12/05/2026	1.0.2	correcciones de bugs de interfaz y feedback reunida por parte de usuarios(gran actualización del sistema)	
18/06/2026	1.2	prototipo final (ultima adición, y sistema funcional y modular)	

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.1. ESTILO ARQUITECTONICO ADOPTADO (EJ. MONOLITICO, MICROSERVICIOS, SOA, CAPAS)	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
3.3. PATRONES DE DISEÑO DE APLICACIÓN (EJ. PATRON MVC, REPOSITORIO, ETC)	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7

4.5.2. Diagrama de despliegue	7
4.5.3. Descripción diagrama de despliegue	7
5. REQUISITOS DE CALIDAD	7
5.1. PROPÓSITO	7
5.3. Reglas y criterios de evaluación de calidad	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. Propósito	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El edificio "El Mirador" se encuentra ubicado en una zona céntrica de la ciudad, lo que implica un mayor valor comercial y una mayor demanda de servicios, así como también un mayor costo de vida y una mayor complejidad en la gestión. El edificio cuenta con 20 pisos, lo que significa una gran cantidad de departamentos y residentes, lo que aumenta la complejidad de la administración y la logística de los servicios comunes.

Actualmente, del total de 160 departamentos el 60 % corresponde a propietarios y el 40% restante corresponde a régimen de arriendo, lo que genera una dinámica en la comunidad, con diferentes intereses y necesidades entre los residentes.

El edificio cuenta con 3 personas encargadas de la mantención y 2 conserjes quienes están a cargo de la seguridad, mantenimiento de las instalaciones y la atención a los residentes. Sin embargo, la cantidad de personal en relación con el número de departamentos y residentes podría ser insuficiente para atender, adecuadamente, las necesidades del edificio, especialmente en caso de imprevistos o demandas extraordinarias

1.2. Propósito

- corregir la ineficiencia en la gestión de cobros y la morosidad que afectan las finanzas del edificio

- automatizar los procedimientos manuales con el fin de aumentar la eficiencia de los mismos

- crear un sistema de registros que permita identificar deudas y deudores a fines al edificio

- abrir canales de comunicación intuitivos para residentes y administrativos del edificio

1.3. Ámbito

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>EFT</i>	Examen final transversal
<i>RN/RNF</i>	Requerimientos Funcionales (qué hace el sistema) y No Funcionales (cómo lo hace).
<i>API</i>	interfaz que permite conectar nuestro sistema con aplicaciones externas, como los bancos.
<i>AWS</i>	Plataforma de servicios en la nube sugerida para el despliegue de la solución tecnológica.

1.5. Resumen ejecutivo

Este informe detalla el diseño de software para modernizar la gestión del edificio "El Mirador" (20 pisos, 160 departamentos). Con un personal muy reducido (2 conserjes y 3 encargados de mantención), los procesos manuales actuales provocan ineficiencia, alta morosidad y desconfianza. La solución implementa una plataforma digital centrada en el

Subsistema de Pagos y Transacciones para automatizar la recaudación, asegurar la liquidez diaria y aliviar la carga operativa.

1.6. Arquitectura del sistema

El sistema utiliza un Estilo Arquitectónico en Capas que independiza la interfaz, la lógica y los datos. En este entorno modular, el Subsistema de Pagos (Subsistema 4) recibe desde el *Subsistema Financiero (Subsistema 3)* los montos pendientes por cobrar. Una vez procesada y validada la transacción de forma segura, se conecta con el *Subsistema de Comunicaciones (Subsistema 5)* para despachar de inmediato el comprobante en PDF al residente.

(ej. vista de escenario, vista lógica, vista de desarrollo, vista de proceso, vista física)

2. VISIÓN DEL SISTEMA

2.1- Descripción general del sistema

El sistema de gestión del edificio "El Mirador" es una plataforma web y móvil integral diseñada para digitalizar la vida en comunidad. El sistema permite centralizar los datos de copropietarios y arrendatarios , procesar solicitudes de mantenimiento y auditar el uso de los fondos comunes.

En lo que respecta al **Subsistema 4 (Pagos y Transacciones)**, la plataforma actúa como una pasarela e historial financiero interno. Permite a los residentes mitigar sus deudas de gastos comunes ordinarios y extraordinarios de forma remota a través de canales digitales bancarios. Al mismo tiempo, proporciona a la administración herramientas automatizadas para la conciliación de transferencias manuales y la gestión del flujo de caja diario.

2.2 Objetivos del sistema

Automatizar la recaudación: Eliminar los procesos manuales en la recepción, validación y registro de los pagos efectuados por los residentes.

Reducir la morosidad: Agilizar los canales de pago e implementar notificaciones de cobro oportunas para mejorar de forma drástica la liquidez y estabilidad financiera del edificio.

Garantizar la transparencia: Proveer acceso en tiempo real a copropietarios, residentes y administradores sobre el estado de cuenta y destino de los pagos de las cuotas.

Optimizar la Eficiencia Administrativa: Liberar tiempo del limitado personal disponible (3 encargados de mantenimiento y 2 conserjes) para que puedan enfocarse en labores estratégicas y operativas críticas.

2.3 Requerimientos funcionales

RF-10 (Registro de Pagos): El sistema debe registrar automáticamente los pagos validados por las pasarelas en línea y permitir al administrador ingresar manualmente los pagos recibidos por transferencia bancaria o caja chica.

RF-11 (Medios de Pago e Integración): El sistema debe ofrecer integración con múltiples medios de pago electrónicos (Tarjetas de Crédito, Débito, Webpay/Flow) y manejar la tipificación de transacciones bancarias.

RF-11.2 (Emisión de Comprobantes): El sistema debe generar de forma automática un comprobante o recibo digital de pago en formato PDF, inalterable, una vez confirmada la transacción.

RF-11.3 (Asociación a Deuda/Documento): El sistema debe rebajar automáticamente el monto pagado del saldo deudor del "Estado de Cuenta" de la propiedad respectiva, asignándole al período correspondiente (gasto común del mes, fondo de reserva o cuota extraordinaria).

2.4 Requerimientos no funcionales

Seguridad y Cifrado: Toda operación transaccional viajará protegida bajo protocolos seguros (HTTPS con TLS 1.3). El sistema cumplirá con las normativas PCI-DSS, lo que significa que la plataforma del edificio jamás guardará contraseñas bancarias ni números de tarjetas en sus servidores.

Disponibilidad Continua: Al tratarse de una plataforma de pagos, el módulo de recaudación debe estar disponible para los copropietarios las 24 horas del día, con una tasa de operatividad del 99.5% anual.

Resistencia a la Concurrencia: El sistema debe estar diseñado para soportar múltiples transacciones en paralelo sin saturarse, especialmente durante los días de vencimiento del gasto común, garantizando que no existan cobros duplicados en los bancos.

Facilidad de Uso (Usabilidad): El flujo para realizar un pago debe ser extremadamente limpio y responsivo, permitiendo que cualquier persona, sin importar su edad o familiaridad con la tecnología, complete la transacción en unos pocos clics desde su teléfono o computador.

2.5 Supuestos y dependencias

Supuesto de Adopción Tecnológica: Se asume que la comunidad (tanto el 60% de propietarios como el 40% de arrendatarios) cuenta con acceso básico a internet y está dispuesta a utilizar canales digitales para abandonar progresivamente el uso de efectivo o cheques.

Supuesto de Inversión Inicial: Se asume que la junta de vigilancia o el comité del edificio autorizará el presupuesto necesario para costear las comisiones e integraciones de las pasarelas de pago digitales elegidas.

Dependencia de Datos (Subsistema 3): El Subsistema de Pagos depende al 100% de la estabilidad del *Subsistema Financiero*. Si este último no genera los cálculos de cobro o el prorrateo a tiempo, el módulo de pagos se quedará sin información base para mostrar los saldos pendientes a los usuarios.

Dependencia Proveedores Externos: Existe una dependencia crítica respecto a la estabilidad de los servidores de las entidades financieras y pasarelas de pago contratadas. Si el proveedor del servicio bancario sufre una caída general, la recaudación en línea del edificio se interrumpirá momentáneamente, requiriendo planes de contingencia para pagos manuales.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para el subsistema de pagos y transacciones (4) Se adoptó un estilo por un estilo arquitectónico por capas combinado con un enfoque de microservicios o servicios desacoplados para la comunicación con pasarelas de pago externas.

Las principales capas del sistema serían:

Capa de Presentación (Interfaz de Usuario): Interfaces intuitivas web y móviles para residentes y administradores.

Capa de Lógica de Negocio (Aplicación): Procesamiento de transacciones, validación de montos, reglas de negocio de asignación de deudas y comunicación con entidades financieras.

Capa de Acceso a Datos (Persistencia): Repositorios para el almacenamiento seguro del estado de las transacciones, históricos de pago y auditorías

3.2. Justificación del estilo según el contexto del sistema

La arquitectura en capas se eligió para garantizar seguridad, orden y continuidad en la gestión de pagos de la comunidad “El Mirador”. Al separar la lógica de negocio del acceso a datos, se protege la información financiera de los residentes y se facilita el cumplimiento de normas de seguridad. Además, esta estructura permite que el sistema crezca fácilmente, integrando nuevas pasarelas de pago como Webpay, Flow o Khipu sin modificar el núcleo de la aplicación. También asegura que, si un servicio externo falla, las funciones esenciales del portal sigan funcionando normalmente.

3.3. Patrones de diseño aplicados (

El subsistema utiliza tres patrones principales:

MVC (Model-View-Controller): organiza la interacción entre usuario, datos y vistas, permitiendo actualizar automáticamente comprobantes y estados de cuenta tras un pago.

Repositorio (Repository): centraliza y simplifica el acceso a la base de datos, separando las operaciones financieras de la lógica técnica.

Estrategia (Strategy): permite manejar distintos métodos de pago bajo una misma estructura, facilitando agregar nuevas opciones sin afectar el sistema principal

MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

Modelar los aspectos funcionales del subsistema de pagos a través de las interacciones de los usuarios clave del Edificio El Mirador , asegurando que se cumplan las necesidades de automatización, reducción de morosidad y transparencia.

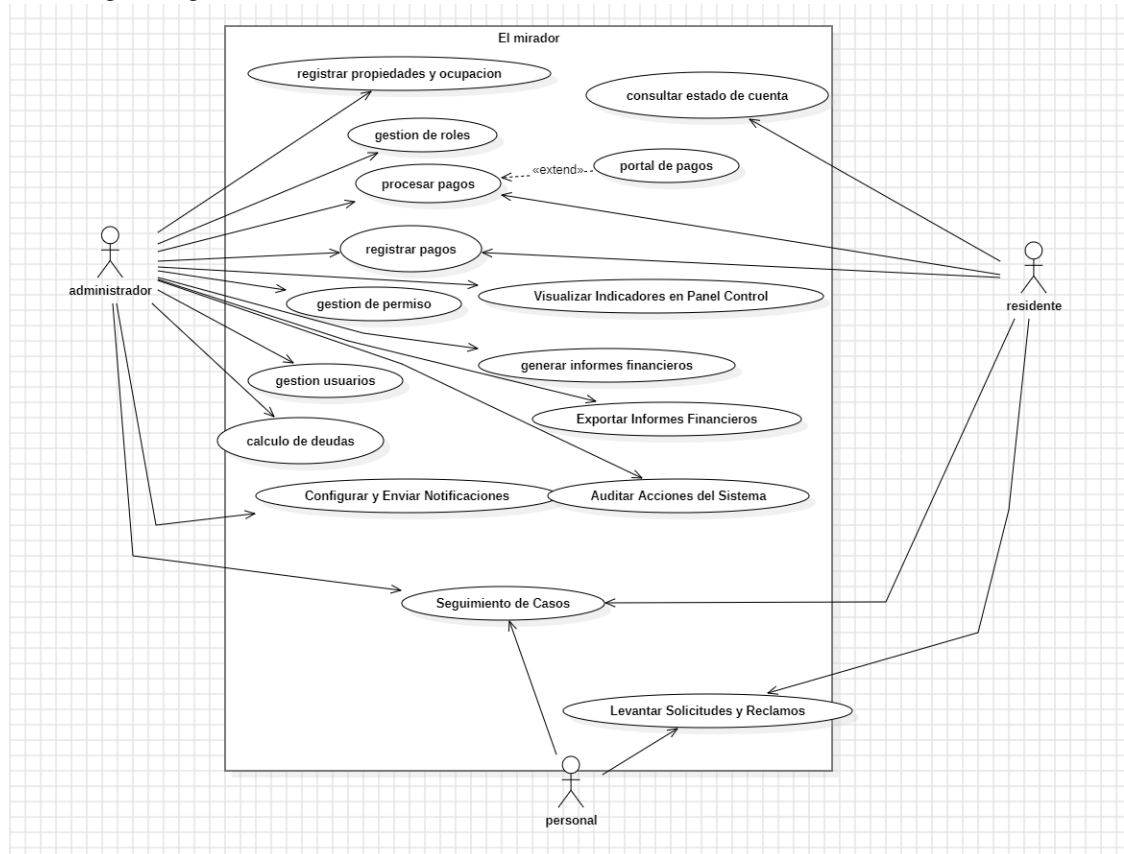
4.1.2. Actores

El Residente se conecta con los casos de uso: *Visualizar Estado de Cuenta, Pagar Gasto Común en Línea y Descargar Comprobante de Pago.*

El Administrador se conecta con: *Registrar Pago Manual, Conciliar Transacciones y Emitir Reporte de Recaudación.*

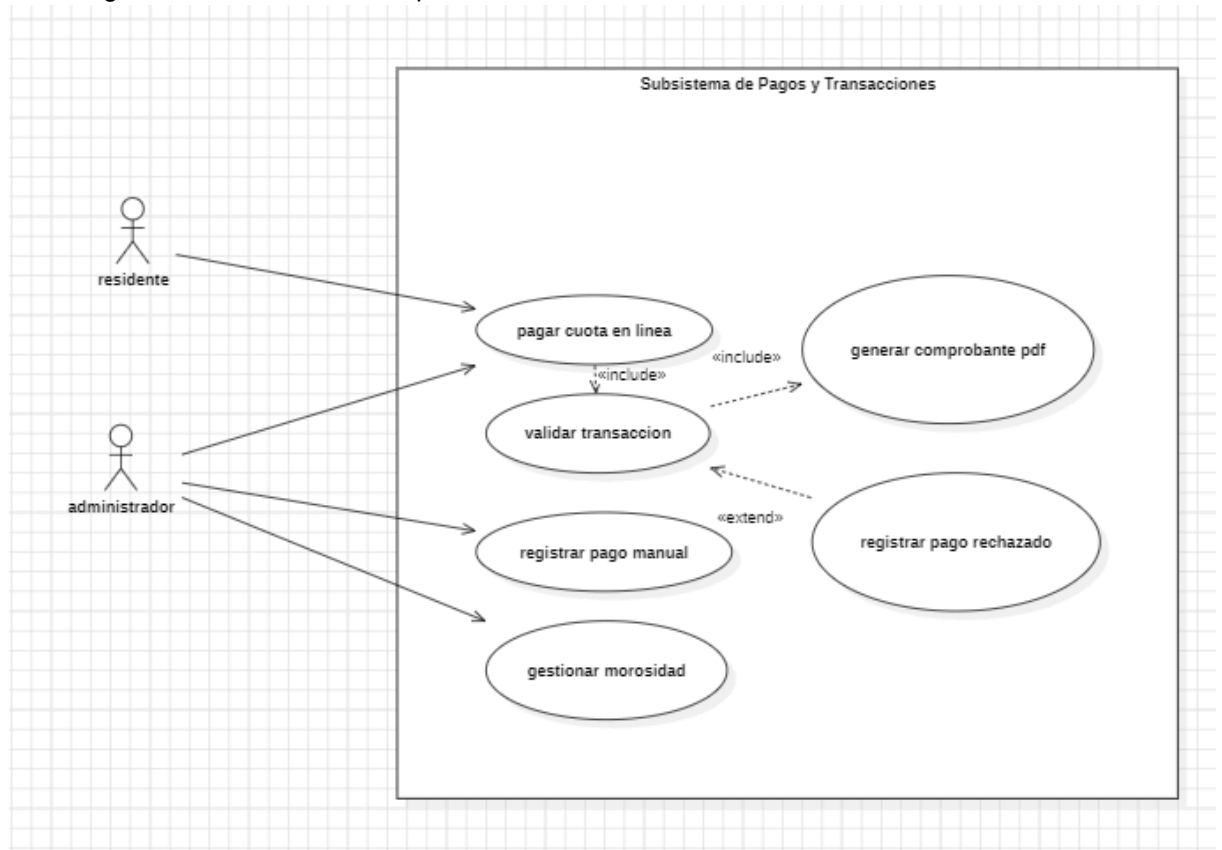
La Pasarela de Pagos interactúa directamente en el proceso de *Pagar Gasto Común en Línea* para validar los fondos.

4.1.3. Diagrama general de casos de uso



4.1.4.

4.1.5. Diagrama de casos de uso específicos



4.1.6. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	comunidad
CU-002-001	Exportar actividades	gerencia y comunidad
CU-002-002	Importar deuda vencida por PDV	Gerencia
CU-004-001	Generación Archivo PDA Importación	Gerencia
CU-004-002	Sistema de reporte, denuncias quejas y soporte	Personal mantenimiento
CU-005-001	Base de datos residenciales	Gerencia

4.1.7. Especificación de casos de uso

Caso de Uso	Pago electrónico de gastos comunes	Identificador: subsistema 4
--------------------	------------------------------------	---------------------------------------

Actores	Residente (Propietario o Arrendatario), Pasarela de Pagos Externa.
Tipo	Primario y esencial (de cara al usuario).
Referencias	RF-10 (Registro de pagos), RF-11 (Integración de medios de pago).
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]
Postcondición	El estado de cuenta se actualiza a "Pagado", se genera el recibo PDF y se dispara la notificación al residente.
Descripción	Este proceso describe cómo el usuario liquida su deuda de gastos comunes de forma 100% digital, asegurando que el dinero se asocie correctamente a su departamento sin que el administrador deba intervenir manualmente.
Resumen	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Actor	[El Residente ingresa a la sección "Mis Pagos" dentro del portal del edificio.
2	Sistema	Busca en la base de datos los saldos pendientes asociados al departamento del usuario y los despliega en pantalla.
3	Actor	Selecciona la cuota a pagar y elige el medio de pago (Débito/Crédito).
[Se describe el proceso o secuencia de pasos ejecutados usando frases cortas] [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].		

CURSO ALTERNATIVO

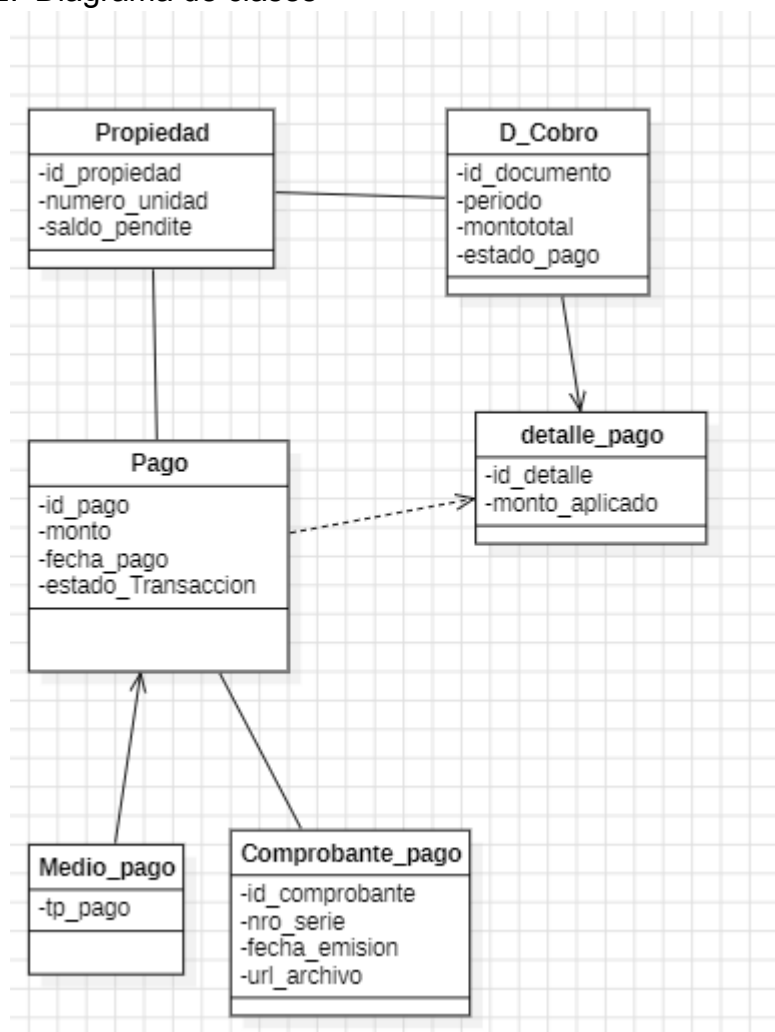
Nro.	Descripción de acciones alternas
5.1	Transacción Rechazada: El banco o la pasarela informan que el pago no pudo procesarse (por falta de fondos o error del usuario). El sistema vuelve al paso 3, muestra un mensaje de error y permite reintentar.
6.1	Fallo en Registro Local: Si el pago se hizo en el banco pero falla la conexión con la base de datos del edificio, el sistema marca la transacción como "Pendiente de Conciliación" para que el administrador la valide manualmente más tarde.
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

El propósito de esta vista para el Subsistema N°4 es modelar la estructura estática y las abstracciones orientadas a objetos que permiten gestionar el ciclo de vida de los flujos financieros entrantes en el Edificio "El Mirador". Su objetivo principal es asegurar la integridad transaccional mediante la validación rigurosa de las reglas de negocio, permitiendo el registro auditable de los pagos realizados por los usuarios, la administración de múltiples medios de pago, la emisión automatizada de comprobantes y la vinculación inequívoca de cada transacción hacia una propiedad y su respectivo documento de cobro o deuda del período.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

Propiedad

Representa la unidad física (departamento) dentro de la comunidad. Aunque pertenece conceptualmente al Subsistema N°1, en este ámbito actúa como el eje central receptor del abono financiero.

Pago

Valor monetario de la operación teniendo en cuenta la fecha, el estado y el monto del mismo

.- MedioPago

Modela y parametriza las distintas vías de recaudación financiera que el sistema soporta

.- DocumentoDeCobro

Proviene del Subsistema Financiero (N°3). Representa la obligación financiera (Gastos Comunes o Gastos Extraordinarios) emitida para un período específico. Es la entidad contra la cual se amortiza el dinero.

.- DetallePago

Clase asociativa que actúa como puente intermedio. Permite resolver el escenario donde un único pago masivo se utiliza para saldar múltiples deudas vencidas o documentos de cobro cruzados de una misma propiedad.

.- ComprobantePago

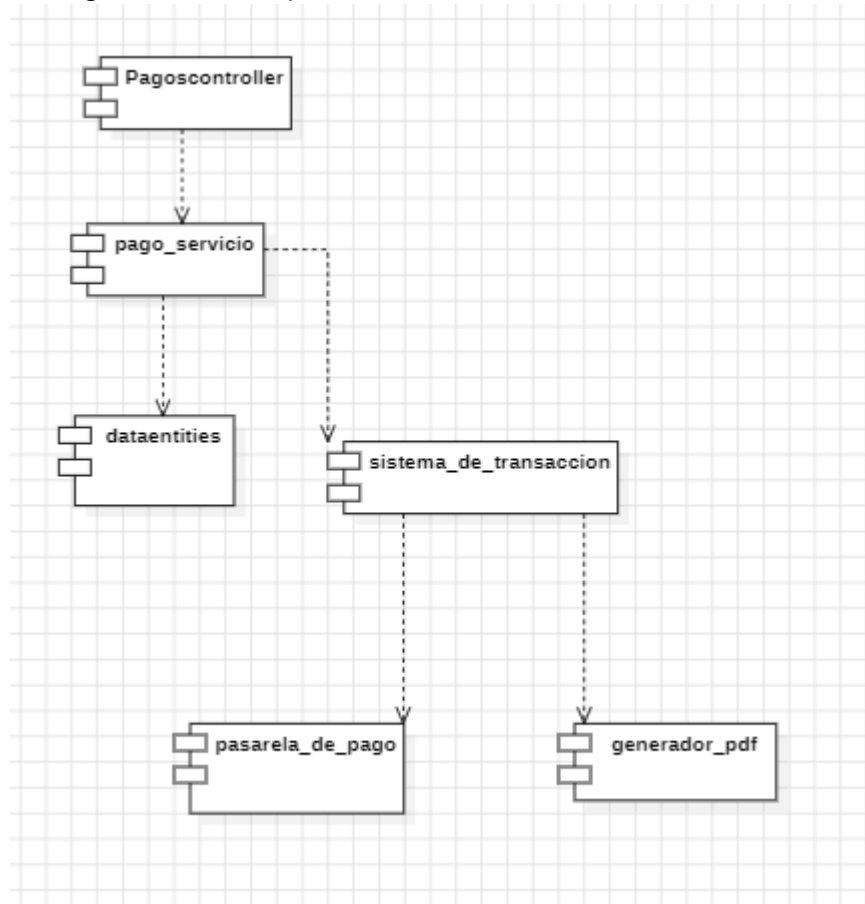
Documento digital oficial de respaldo emitido de forma obligatoria tras la aprobación de la transacción, garantizando la transparencia exigida en los objetivos del negocio.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El propósito de esta vista es definir la estructura física del código fuente y los artefactos de software que componen el Subsistema de Pagos y Transacciones. Permite a los desarrolladores y arquitectos comprender cómo se agrupan las clases lógicas en paquetes físicos, cómo interactúan los componentes en tiempo de compilación y desarrollo, y qué dependencias existen con librerías de terceros. Esto asegura el cumplimiento de los requerimientos de modularidad y seguridad.

4.3.2. Diagrama de componente



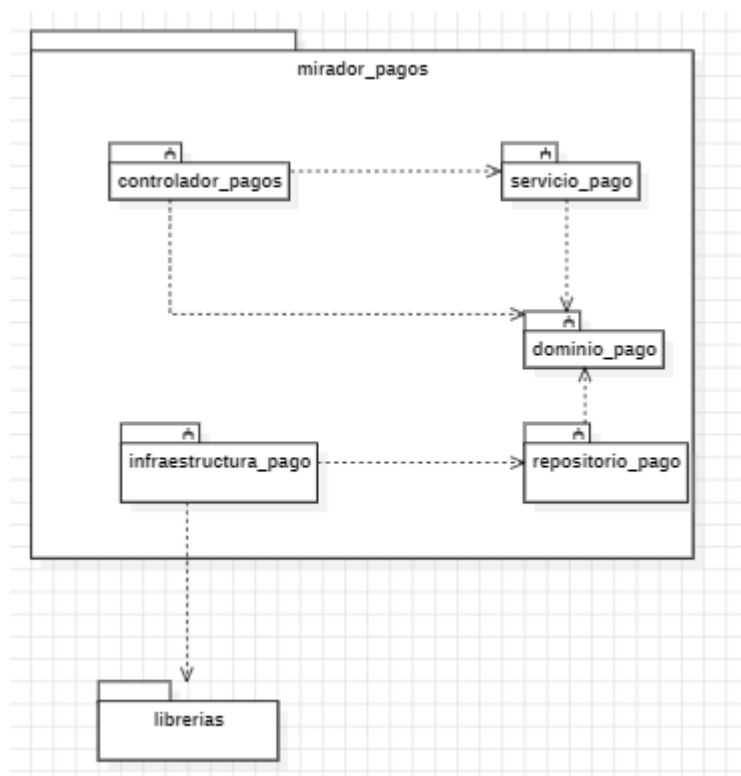
4.3.3. Descripción diagrama de componente

El diagrama representa los bloques de construcción físicos independientes que se compilan y despliegan en el sistema:

- **PagosController:** Módulo ejecutable encargado de recibir las solicitudes de pago de la interfaz web o móvil (**RNF-10**). Actúa como la fachada de entrada del subsistema.
- **PagosService:** El motor principal de la aplicación. Contiene los componentes binarios que aplican las auditorías, orquestan el flujo del dinero y validan los estados de las deudas.

- **DataEntities:** Componente reutilizable que empaqueta las estructuras de datos compartidas por el sistema (tablas mapeadas a objetos). Es utilizado por el servicio para persistir la información.
- **Sistema_de_transaccion:** Componente especializado en la comunicación con servicios externos. Aísla los aspectos de red y protocolos técnicos del resto de la aplicación corporativa.
- **Pasarela_de_Pago. / Generador_pdf (Componentes Externos): ***
Pasarela_de_Pago representa el conector físico (módulo de software provisto por terceros) para el procesamiento de tarjetas bancarias en tiempo real.
 - Generador_pdf es el componente encargado de tomar los datos de la transacción exitosa y renderizar el archivo físico binario que se convertirá en el Comprobante de Pago descargable para el residente (**RF-29**).

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El código fuente del subsistema se organiza siguiendo un patrón de arquitectura limpia o por capas dentro del paquete raíz `cl.mirador.pagos`:

- **mirador_pagos** : Contiene los componentes que exponen los endpoints o interfaces de entrada (ej. controladores REST) para que los residentes y administradores inicien o consulten transacciones. Depende directamente de la capa de servicios.

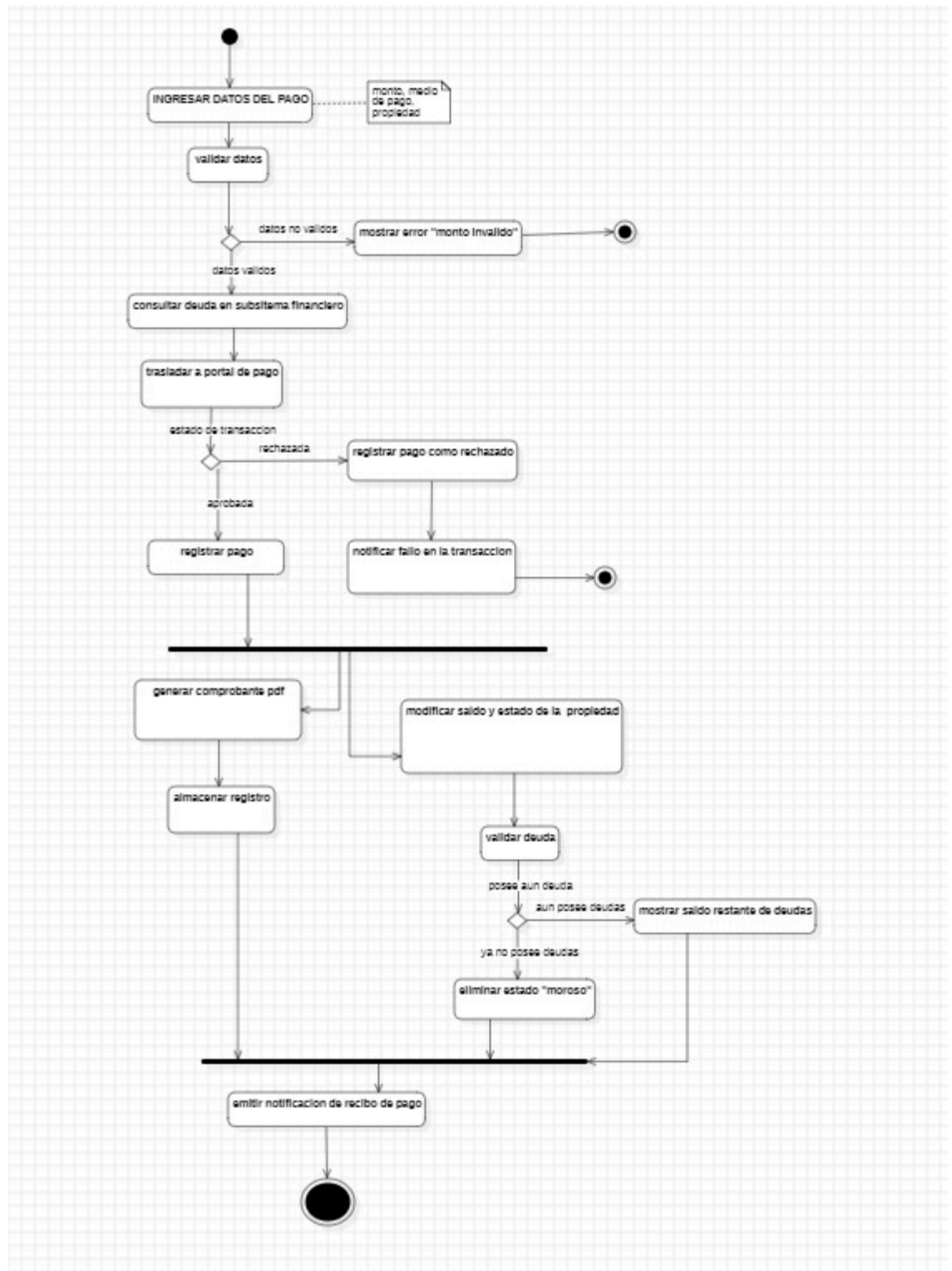
- **servicio_pago(mirador_pagos):** Aquí se implementan las reglas como **RN-08** (validar que el monto sea mayor a cero) y se coordina el flujo transaccional. Es el puente entre los controladores y el acceso a datos.
- **dominio_pago(mirador_pagos):** Contiene el núcleo lógico (las clases lógicas puras como Pago, ComprobantePago y DetallePago). Todas las capas del subsistema conocen y dependen de este paquete.
- **repositorio_pago(mirador_pagos):** Define las interfaces de almacenamiento encargadas de guardar, actualizar y consultar los registros de pagos y transacciones en la base de datos.
- **infraestructura_pago(mirador_pagos):** Contiene las implementaciones técnicas concretas para interactuar con el exterior, cumpliendo con la interoperabilidad (**RNF-13**). Es el paquete que se comunica de forma directa con las **Librerías Externas** (SDKs de pasarelas de pago externas o librerías de generación de PDFs).

4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de la Vista de Procesos para este subsistema es modelar el comportamiento dinámico y el flujo de ejecución concurrentes que ocurren cuando se procesa un pago dentro del Edificio "El Mirador". Esta vista valida cómo las interacciones en tiempo de ejecución garantizan la sincronización de datos, el manejo de errores transaccionales, el cumplimiento estricto de las reglas de negocio (como impedir montos menores o iguales a cero — **RN-08**) y la actualización asíncrona o síncrona de los estados de deuda y generación de comprobantes, asegurando el rendimiento (**RNF-03**) y la confiabilidad (**RNF-08**) del sistema.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El proceso describe paso a paso el ciclo de vida de la ejecución del hilo transaccional basándose en los requerimientos y reglas provistos:

A. Etapas del Flujo de Trabajo

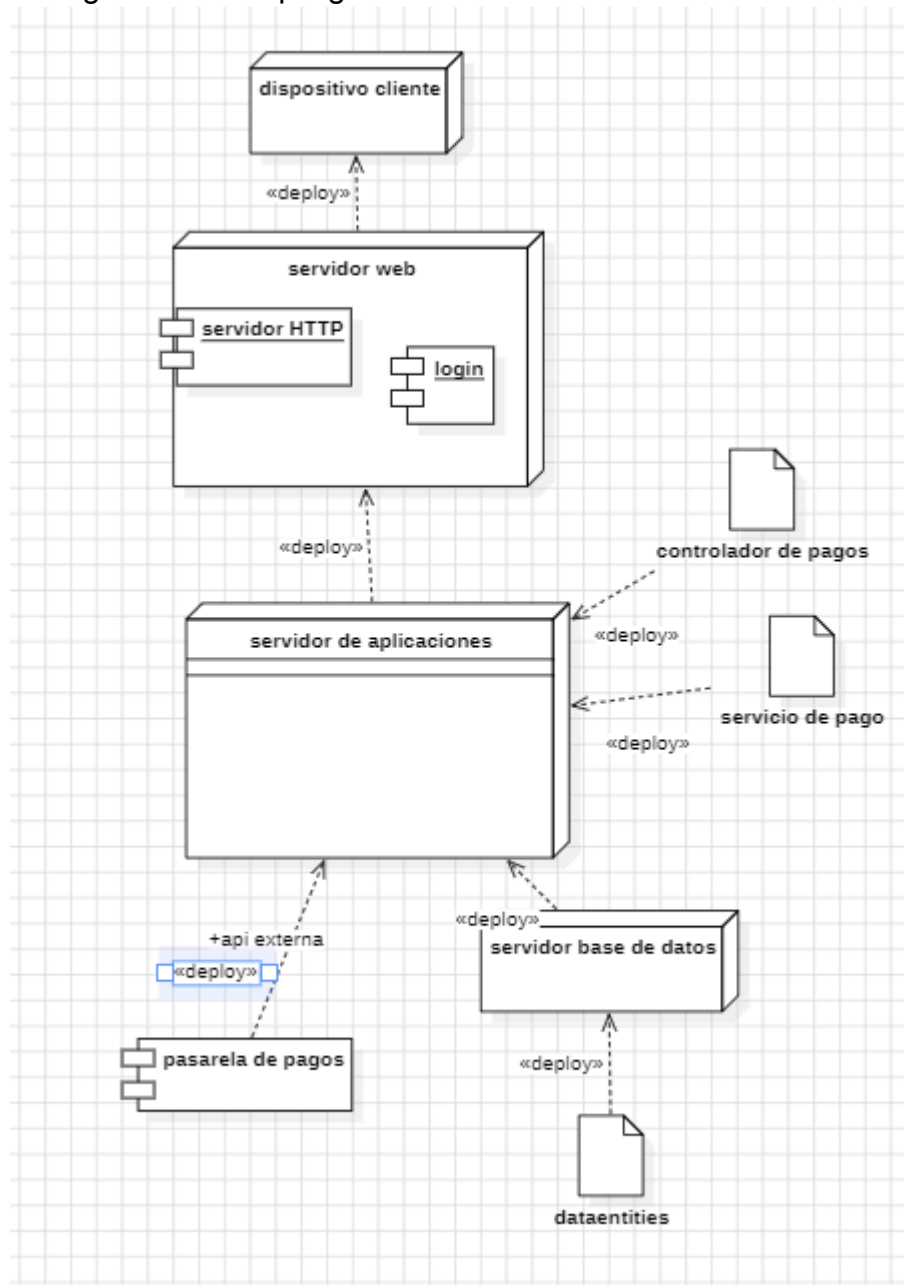
1. **Ingreso y Validación Inicial:** El usuario interactúa con la interfaz para enviar el monto, la propiedad seleccionada y el medio de pago. Lo primero que ejecuta el sistema es la verificación de la calidad de la información (**RNF-16**). Aplicando la regla **RN-08**, si el monto es ≤ 0\$, el flujo se interrumpe inmediatamente arrojando una excepción visual y bloqueando la persistencia.
2. **Vinculación con la Obligación Financiera:** El proceso se conecta dinámicamente con el *Subsistema Financiero* (N°3) para recuperar las deudas vigentes o documentos de cobro pendientes asociados a esa propiedad específica (**RN-09**), calculando cómo se prorrateará o aplicará el abono entrante.
3. **Procesamiento de la Transacción:** El sistema delega la operación al conector de la pasarela de pagos externa o procesa el flujo de revisión de la transferencia bancaria. Si el servicio externo falla o es rechazado, se escribe el estado como "Rechazado" en la bitácora de auditoría (**RNF-07**) y se alerta al usuario.
4. **Bifurcación Concurrente (Fork):** Una vez que la transacción es confirmada como exitosa, el sistema abre dos hilos de ejecución paralelos para optimizar los tiempos de respuesta y no degradar la experiencia de usuario (cumpliendo con el rendimiento de **RNF-03**):
 - **Hilo de Actualización de Saldos:** Modifica los balances financieros de la propiedad. Si el pago cubre la totalidad de la deuda vencida, el proceso ejecuta automáticamente un disparador (trigger) para cambiar el estado de la propiedad de "Morosa" a "Al día", resolviendo la lógica de la regla **RN-07**.
 - **Hilo de Generación de Artefactos:** El sistema de manera asíncrona invoca el servicio de renderizado de documentos para construir el archivo PDF del comprobante (**RF-29**) y guardar su URL en la base de datos de manera consistente.
5. **Sincronización y Cierre (Join):** Cuando ambos hilos paralelos terminan satisfactoriamente, el flujo se unifica de nuevo. Como última actividad, se dispara un evento hacia el *Subsistema de Comunicaciones* (N°5) para despachar de forma automática la notificación por correo electrónico o mensajería al residente con el comprobante adjunto, garantizando el registro en el historial de comunicaciones (**RF-17**).

4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de la Vista Física para este subsistema es modelar la infraestructura de hardware y comunicaciones sobre la cual se ejecutarán de forma real los componentes de pago. Su objetivo primordial es mapear la distribución de la carga de procesamiento, garantizar la alta disponibilidad del 99% mensual (**RNF-02**), asegurar los tiempos de respuesta menores a 3 segundos (**RNF-03**), e implementar las restricciones de seguridad perimetral y cifrado de datos necesarios para proteger la información financiera y transaccional crítica de la comunidad (**RNF-05**, **RNF-06**).

4.5.2. Diagrama de despliegue



4.5.3.Descripción diagrama de despliegue

La topología de infraestructura está diseñada para soportar los atributos de calidad exigidos, desglosándose en los siguientes nodos físicos/virtuales y canales de conexión:

A. Catálogo de Nodos e Infraestructura

1. **Nodo Dispositivo de Usuario:**
 - **Descripción:** Representa el hardware del residente (celular, tablet o computador) ejecutando navegadores modernos. No requiere instalación local para las consultas en línea, garantizando la accesibilidad multiplataforma.
2. **Nodo Servidor Web:**
 - **Descripción:** Es la primera línea de defensa y el punto de entrada a la infraestructura del edificio. Se encarga de terminar las conexiones seguras y distribuir las peticiones concurrentes para asegurar la escalabilidad (**RNF-04**) y el soporte multiusuario sin degradación del servicio (**RNF-17**).
3. **Nodo Servidor de Aplicaciones:**
 - **Descripción:** Entorno informático cerrado donde reside la inteligencia del software.
 - **Artefactos Desplegados:** Aloja físicamente los módulos controlador de pagos y servicio de pagos . Al concentrar la ejecución en este nodo de manera modular, se agilizan las operaciones lógicas críticas (como la validación del monto mayor a cero — **RN-08**) reduciendo la latencia de respuesta para cumplir el estándar de menos de 3 segundos (**RNF-03**).
4. **Nodo Servidor de Base de Datos:**
 - **Descripción:** Servidor dedicado exclusivamente al almacenamiento persistente, aislado en una subred privada.
 - **Artefactos y Seguridad:** Ejecuta el esquema estructurado derivado de 'DataEntities'. Implementa políticas de respaldo automático y recuperación inmediata (**RNF-09**) para evitar pérdidas de información ante fallos de energía (**RNF-08**), resguardando bajo cifrado la confidencialidad de los montos recaudados (**RNF-06**).
5. **pasarela de pagos(api externa):**
 - **Descripción:** Plataforma en la nube operada por el proveedor financiero externo. No se encuentra dentro de los servidores del edificio, sino que se consume como un servicio distribuido por internet de alta disponibilidad.

B. Protocolos y Canales de Comunicación (Red)

- **Cliente a Servidor Web :** Canal público encriptado con TLS 1.3. Cifra los datos bancarios y personales desde el navegador del residente, garantizando la seguridad de acceso (**RNF-05**).
- **Servidor Web a Servidor de Aplicaciones :** Canal de comunicación de alta velocidad corriendo dentro de una red privada virtual (VPC). Al estar aislado de internet, optimiza el rendimiento sin comprometer la seguridad.
- **Servidor de Aplicaciones a Base de Datos :** Conexión interna dedicada para persistir las transacciones aprobadas, la morosidad y las deudas. Cuenta con mecanismos para tolerar de forma concurrente el acceso simultáneo de múltiples usuarios administrativos y residentes (**RNF-17**).
- **Servidor de Aplicaciones a Pasarela Externa:** Enlace seguro de interoperabilidad exterior (**RNF-13**). Envía las solicitudes de cobro cifradas y recibe la confirmación de la transacción en tiempo real para autorizar inmediatamente el cambio de estado de la propiedad (**RN-07**)

5. REQUISITOS DE CALIDAD

5.1. Propósito

Establecer las directrices operacionales no funcionales y de seguridad necesarias para que el Subsistema de Pagos y Transacciones funcione con

excelencia y se adapte al contexto del edificio El Mirador.

5.2. Atributos de calidad Usabilidad , Accesibilidad (WCAG) Rendimiento, Seguridad

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN
Usabilidad	Facilidad con la administradores y residentes pueden usar el sistema (interfaz intuitiva	Hay distintos tipos de usuarios (administrador, conserjes, residentes). Si el sistema es complejo, no se utilizará correctamente, lo que empeoraría los problemas actuales de gestión y comunicación.
Transparencia	Capacidad del sistema para registrar y mostrar claramente todas las transacciones, pagos y gastos.	Uno de los problemas principales del caso es la desconfianza. Este atributo es clave para mejorar la relación entre residentes y administración.
Rendimiento	El sistema debe responder rápidamente ante consultas,pagos y generación de reportes	Procesos como consultas de deudas, generación de informes o registro de pagos no pueden ser lentos, ya que afectaría la eficiencia administrativa y la experiencia del usuario.
Seguridad	Protección de datos personales y financieros mediante autenticación, autorización y cifrado	Se manejarán datos sensibles (pagos, deudas, información personal). Un fallo de seguridad afectaría la confianza de los residentes y podría tener consecuencias legales.

5.3. Reglas y criterios de evaluación de calidad

Seguridad de los Datos Financieros (Confidencialidad): Ningún dato sensible de las tarjetas de crédito o cuentas bancarias de los residentes debe guardarse en los servidores locales del edificio. El subsistema debe delegar la captura de estos datos exclusivamente en plataformas con certificación PCI-DSS a través de la pasarela.

Disponibilidad del Sistema: El componente de cara al residente para efectuar los pagos web debe registrar una disponibilidad mínima del 99.5% anual.

Integridad Transaccional (No-Repudio y Consistencia): El sistema debe implementar un mecanismo de transacciones atómicas (ACID). Si el sistema procesa un cobro de forma exitosa en el banco pero falla al guardar el registro en la base de datos local, se debe gatillar un *rollback* o un proceso automático de conciliación para evitar cobrar al residente sin abonar la deuda.

Tiempos de Respuesta: El procesamiento local de la validación del token de transacción y la generación del comprobante en formato PDF no debe superar los 2.5 segundos por requerimiento.

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6

6.1. Propósito

La idea de esta sección es justificar las buenas prácticas que usamos para que el código del sistema de pagos no sea un caos. Básicamente, aplicamos estos principios para que el software sea fácil de mantener, que si a futuro queremos meter otra pasarela de pago (como Flow o Khipu) no tengamos que romper todo lo que ya funciona, y que cada parte del código haga una sola cosa bien. En resumen: ordenar la estructura para que sea modular y cumpla con lo que nos piden en la carrera.

6.2. Principios de diseño:

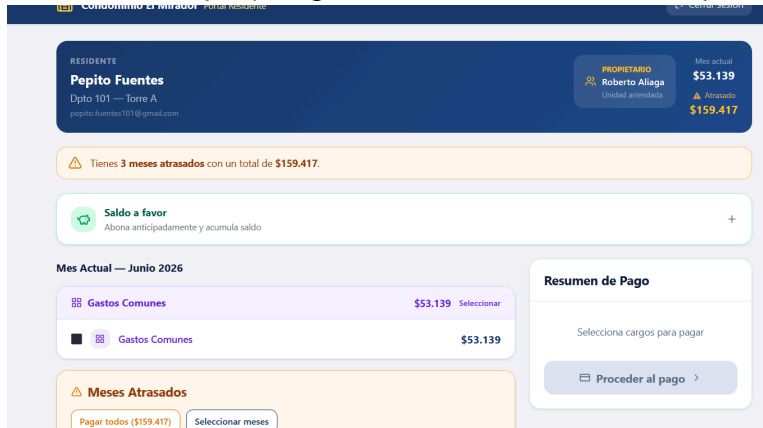
PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones
Abstracción	Consiste en ocultar los detalles de implementación complejos, centrándose únicamente en lo que hace un sistema o componente y no en cómo lo hace .	Interfaz de usuario simplificada, sistemas de notificaciones.
Acoplamiento	Se divide en 2 tipos, alto y bajo el alto tiene mucha dependencia en cambio el acoplamiento bajo no	circuitos cerrados, generadores de respaldo y respaldo en servicios de cloud externos.
Encapsulamiento		
Modularidad	Es la capacidad de	

7. PROTOTIPO

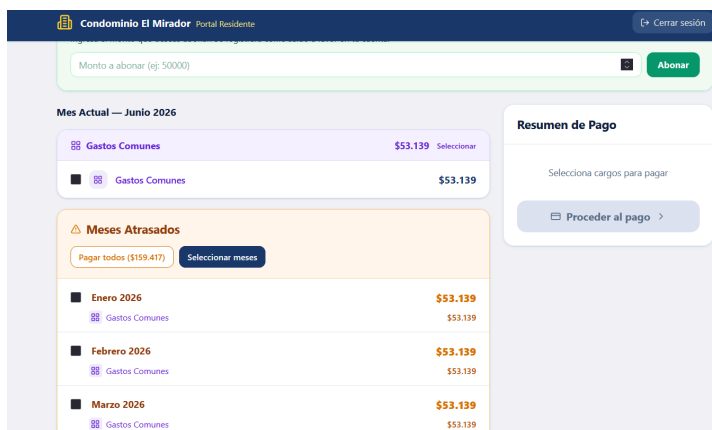
7.1. Propósito

El prototipo de alta fidelidad desarrollado para la plataforma web "Condominio El Mirador" tiene como propósito fundamental validar de manera visual e interactiva los flujos financieros críticos tanto para los residentes como para la administración antes de iniciar la fase de codificación.

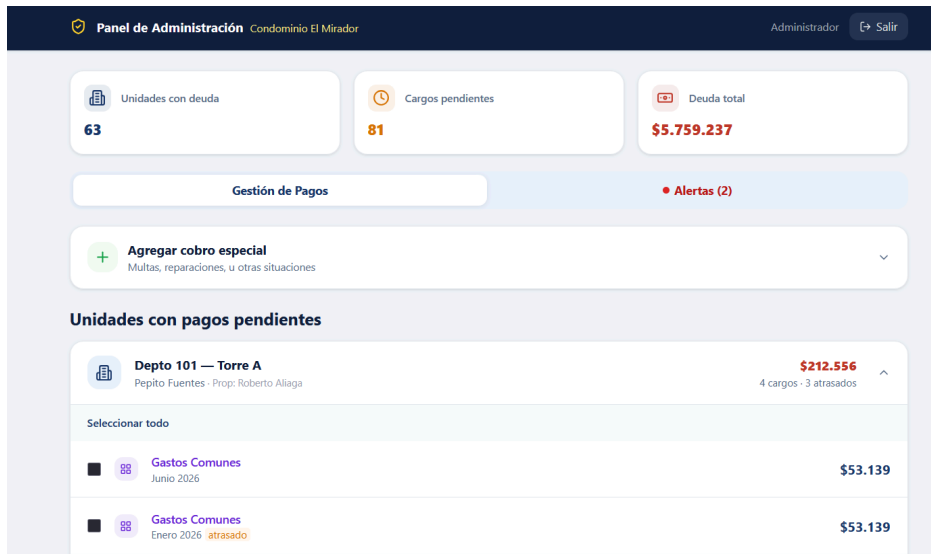
7.2. Mockups (imágenes con una breve descripción)



Esta interfaz representa el panel principal que visualiza el residente al iniciar sesión. Su arquitectura está diseñada para ofrecer visibilidad inmediata de su situación financiera. Destaca un banner superior de advertencia en tono naranja que consolida la cantidad de meses atrasados y el monto total en mora, reduciendo la carga cognitiva. En la zona central se desglosa el cobro del mes actual (Junio 2026), mientras que en el lateral derecho se ancla de forma persistente el módulo interactivo **"Resumen de Pago"**, el cual guía al usuario para proceder a la pasarela transaccional.



Esta vista ilustra el comportamiento interactivo de la plataforma cuando el residente decide realizar un abono anticipado. El componente "Saldo a favor" actúa como un menú colapsable de tipo acordeón que, al expandirse, despliega un campo de texto limpio (Monto a abonar) y un botón de acción directa (Abonar). Esta pantalla simula un estado dinámico donde conviven las deudas históricas desglosadas por mes



Corresponde a la interfaz maestra diseñada para el rol operativo del administrador. En la parte superior se consolida un cuadro de mandos con tres Indicadores Clave de Rendimiento (KPIs) de alto contraste: unidades con deuda, cargos pendientes y la deuda total global acumulada en la comunidad. Cuenta con un componente desplegable para "Agregar cobro especial" y una sección inferior de auditoría detallada por departamento, donde se etiquetan explícitamente los meses en estado "atrasado" para agilizar la gestión de cobranza.

7.3. Justificar herramientas de prototipado

Para la construcción de estos mockups se seleccionó Figma como herramienta principal debido a las siguientes ventajas técnicas:

Diseño basado en componentes y estilos globales: Permite garantizar la consistencia visual a través de átomos y moléculas de UI (botones, campos de texto y tarjetas), facilitando modificaciones globales instantáneas.

Entorno Colaborativo y Prototipado Navegable: Facilita la creación de transiciones y estados dinámicos (como la expansión de menús acordeón) en un entorno en la nube, permitiendo que múltiples stakeholders revisen y simulen los flujos en tiempo real sin necesidad de instalaciones locales.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

Evaluacion_Heuristica_Nielsen.xlsx

8.1. Propósito

El propósito de realizar una evaluación heurística interna basada en los 10 Principios de Usabilidad de Jakob Nielsen es ejecutar una auditoría técnica experta sobre el prototipo de alta fidelidad. Este proceso metodológico permite identificar de forma analítica las violaciones a las reglas de usabilidad, determinar su nivel de gravedad y establecer prioridades de corrección antes de exponer la interfaz a pruebas con usuarios reales o enviarla a la fase de maquetación en código.

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

El propósito del control de versiones aplicado a la documentación y diseño del proyecto es garantizar la trazabilidad histórica, la gobernanza de activos digitales y la integridad de los entregables a lo largo del ciclo de vida del desarrollo. Esta práctica permite auditar qué cambios se realizaron, bajo qué justificación técnica o heurística se aplicaron y previene la pérdida de información o la coexistencia caótica de archivos duplicados dentro del equipo de trabajo.

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial))

Para la gobernanza y evolución del proyecto, se implementó una estrategia de Control de Versiones Semántico (SemVer) bajo la estructura formal **MAJOR.MINOR.PATCH**. Se justificó esta elección debido a que ofrece una comunicación clara y estandarizada sobre la magnitud de cada iteración del software. El ciclo de vida del prototipo quedó documentado en GitHub bajo

9.3. Justificar herramientas de versionamiento

Historial de Versiones Nativo de Figma: Se justificó su uso para el control del lienzo de diseño debido a que permite registrar puntos de restauración nombrados en la nube sobre un único archivo maestro. Esto elimina de raíz la mala práctica de duplicar archivos locales (evitando esquemas de nombres confusos como "*diseño_final_v2_final.fig*"), permitiendo al equipo comparar evoluciones visuales cronológicamente de forma limpia.

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt

ID	Fase / Hito / Tarea	Duración Proyectada	Fecha de Inicio	Fecha de Término	Entregable / Hito Vinculado	Estado
1	FASE DE ANÁLISIS Y PLANIFICACIÓN	4 semanas	01/04/2026	28/04/2026	Hito: Lanzamiento Inicial	Finalizado
1.1	Levantamiento de Requerimientos y definición del contexto del problema (El Mirador).	1.5 semanas	01/04/2026	10/04/2026	Matriz de Requerimientos (RF/RNF).	Finalizado
1.2	Modelamiento del Alcance y Especificación de Casos de Uso del Subsistema de Pagos.	1.5 semanas	11/04/2026	21/04/2026	Casos de Uso (CU-001 a CU-005).	Finalizado
1.3	Definición de Arquitectura en Capas y Patrones de Diseño (MVC, Repositorio, Estrategia).	1 semana	22/04/2026	28/04/2026	Versión 1.0 (Prototipo Inicial Documental)	Finalizado
2	FASE DE DISEÑO Y PROTOTIPADO (SPRINT 1)	2 semanas	29/04/2026	12/05/2026	Hito: Ajuste UX/UI por Feedback	Finalizado
2.1	Diseño de Interfaces de Usuario en Figma (Panel Residente, Abonos y Consola Administrador).	1 semana	29/04/2026	04/05/2026	Mockups de alta fidelidad iniciales (v1.0.1).	Finalizado
2.2	Ejecución de Auditoría Técnica: Evaluación de Calidad Heurística de Nielsen.	4 días	05/05/2026	08/05/2026	Matriz Excel de Evaluación Heurística.	Finalizado
2.3	Levantamiento de correcciones de bugs de interfaz y feedback de usuarios en Figma.	4 días	09/05/2026	12/05/2026	Versión 1.0.2 (Gran actualización del sistema)	Finalizado
3	FASE DE MODELADO TÉCNICO Y ARQUITECTURA (SPRINT 2)	1 semana	13/05/2026	18/05/2026	Hito: Cierre de Línea Base Arquitectura	Finalizado
3.1	Construcción del Modelo Estático: Diagrama de Clases y lógica asociativa (Detalle_Pago).	2 días	13/05/2026	14/05/2026	Vista Lógica (Diagrama de Clases).	Finalizado
3.2	Definición de la Vista de Implementación (Componentes y Paquetes cl.mirador.pagos).	2 días	15/05/2026	16/05/2026	Versión 1.1 (Última Revisión del Informe DAS)	Finalizado
3.3	Modelado Dinámico y Físico (Diagramas de Actividad con Fork/Join concurrente y Despliegue).	2 días	17/05/2026	18/05/2026	Línea Base Aprobada (Próxima revisión).	Finalizado
4	FASE DE DESARROLLO E INTEGRACIÓN (SPRINTS 3 Y 4)	4 semanas	19/05/2026	15/06/2026	Hito: Motor Transaccional Construido	Finalizado
4.1	Programación de la Capa de Datos (DataEntities) y Repositorios de persistencia en subred VPC.	1 semana	19/05/2026	26/05/2026	Base de Datos inicial aislada y segura.	Finalizado
4.2	Desarrollo de la Lógica de Negocio (PagosService) y reglas de validación transaccional (Monto > 0).	1.5 semanas	27/05/2026	05/06/2026	Motor lógico transaccional implementado.	Finalizado
4.3	Integración con APIs externas (Webpay/Flow/Khipu) bajo el patrón Estrategia.	1.5 semanas	06/06/2026	15/06/2026	Módulo de comunicación exterior (infraestructura_pago).	Finalizado
5	FASE DE TESTING, CIERRE Y ENTREGA (SPRINT 5)	3 días	16/06/2026	18/06/2026	Hito: Prototipo Final y Sistema Funcional	Finalizado
5.1	Pruebas de Resistencia a la Concurrencia (Propiedades ACID) y renderizado asíncrono de PDFs.	2 días	16/06/2026	17/06/2026	Reporte de QA / Pruebas unitarias aprobadas.	Finalizado
5.2	Despliegue final en la nube (AWS) y publicación del código modular en GitHub.	1 día	18/06/2026	18/06/2026	Versión 1.2 (Prototipo final y sistema funcional modular).	Finalizado